

CodioProjectFileEditFindViewToolsEducationHelpConfigur.Project I..Configur..

exercise3.pyx

```
1 import requests
2 import nltk
3
4 from googlesearch import search
5 from lxml import html
6 from bs4 import BeautifulSoup
7 from sklearn.feature_extraction.text import TfidfVectorizer
8
9 # helper function to generate text corpus from html elements
10 def generate_corpus(all_p_elements):
11     corpus = ""
12     for p_element in all_p_elements:
13         corpus += '\n' + ''.join(p_element.findAll(text = True))
14     return corpus
15
16 # Collect user input
17 query = input("Enter the query to search on Google: ")
```

29% (11:14)

Terminalx

```
'technology', 'television', 'tennis', 'term', 'terms', 'territories', 'territory', 'test', 'tested', 't
eater', 'theaters', 'theatrical', 'theatrically', 'theme', 'think', 'thought', 'tie', 'tied', 'tier', 'f
me', 'title', 'titled', 'titles', 'tool', 'total', 'track', 'traditional', 'traffic', 'trailer', 'trail
s', 'trailing', 'transit', 'trial', 'trilogy', 'trim', 'turned', 'tweeted', 'twice', 'tying', 'typograph
', 'ultimately', 'unavailable', 'underrepresented', 'undisclosed', 'unified', 'unlike', 'unlimited', 'un
atched', 'unpredictable', 'unscripted', 'upcoming', 'updated', 'upfront', 'urban', 'use', 'used', 'user
'users', 'using', 'valuable', 'variety', 'various', 've', 'ventured', 'venue', 'vice', 'video', 'videos
', 'viewed', 'viewer', 'viewers', 'viewership', 'viewing', 'views', 'violated', 'violating', 'violence',
virtual', 'visited', 'visual', 'visually', 'volume', 'wanted', 'watch', 'watched', 'web', 'website', 'we
k', 'weekend', 'weekly', 'went', 'western', 'wife', 'wiki', 'wikipedia', 'win', 'window', 'winning', 'w
s', 'withdrew', 'witnessing', 'woman', 'won', 'work', 'worked', 'workshops', 'world', 'worldwide', 'wor
', 'writer', 'writers', 'year', 'years']
[[0. 0. 0. ... 0. 0. 0.]
[0. 0. 0. ... 0. 0. 0.]
[0. 0. 0. ... 0. 0. 0.]
...
[0. 0. 0. ... 0. 0. 0.]
[0. 0. 0. ... 0. 0. 0.]
[0. 0. 0. ... 0. 0. 0.]]
codio@exactgemini-sloganalien:~/workspace$
```

Guidex

Collapse

<>⚙️☰

Coding Exercise 3

In the top-left pane, implement a TF-IDF scored bag of words model on the text from the webpage top Google search result of the `query` which is captured from keyboard input.

When creating your model, make sure to:

- Use the `googlesearch` library
- Use Python `requests` library
- Use `lxml`
- Use BeautifulSoup
- Use `nltk.sent_tokenize`
- Use `TfidfVectorizer`
- Filter out punctuation
- Filter out stop words
- Ignore case

Output the (1) top search result (2) vocabulary and the (3) vectors.

Test your code using the terminal in the bottom-left:

TRY IT

Example Input/Output:

netflix wikipedia

https://en.wikipedia.org/wiki/Netflix
['00' '000' '10' ... 'zealand' 'zero' 'zim']
[[0. 0. 0. ... 0. 0. 0.]
[0. 0. 0. ... 0. 0. 0.]
[0. 0. 0. ... 0. 0. 0.]
...
[0. 0. 0. ... 0. 0. 0.]
[0. 0. 0. ... 0. 0. 0.]
[0. 0. 0. ... 0. 0. 0.]]

exercise3

Submit your code by clicking the Check It button.

A solution:

```
# use the google search api to fetch top 3 search results
google_search_results = list(search(query, stop=3, pause=1))

# use the requests api to fetch the top result webpage
webpage = requests.get(google_search_results[0])
webpage_tree = html.fromstring(webpage.content)
webpage_soup = BeautifulSoup(webpage.content, "lxml")

# extract all <p> elements from webpage soup object
all_p_list = webpage_soup.findAll('p')

# generate corpus from all <p> elements
corpus = ""
for p_element in all_p_list:
    corpus += '\n' + ''.join(p_element.findAll(text = True))

# Tokenisation
sentence_tokens = nltk.sent_tokenize(corpus)# converts raw text to list

# Calculate TFIDF matrix
sentence_tokens.append(query)

vectorizer = TfidfVectorizer(stop_words='english')
X = vectorizer.fit_transform(sentence_tokens)

print(google_search_results[0])
print(vectorizer.get_feature_names_out())
print(X.toarray())
```

Check It!

LAST RUN on 12/27/2023, 5:42:27 AM

Check 1 passed

Check 2 passed

Next